

Assignment_6.1

L.Indu

2303a51250

Batch:18

Task Description #1 (AI-Based Code Completion for Loops)

Task: Use an AI code completion tool to generate a loop-based program.

Prompt:

“Generate Python code to print all even numbers between 1 and N using a loop.”

Expected Output:

- AI-generated loop logic.
- Identification of loop type used (for or while).
- Validation with sample inputs.

Prompt:

#Generate Python code to print all even numbers between 1 and N using a loop.

Sample input

```
evenloop.py > ...
1  #Generate Python code to print all even numbers between 1 and N using a loop.
2  def print_even_numbers(n):
3      for i in range(1, n + 1):
4          if i % 2 == 0:
5              print(i)
6
7  # Sample input
8  N = int(input("Enter a number: "))
9  print_even_numbers(N)
10
```

Output:

```
TERMINAL
PS C:\Users\user1\Desktop\python> & C:/Users/user1/AppData/Local/Programs/Python/Python313/python.exe c:/Users/user1/Desktop/python/evenloop.py
Enter a number: 6
2
4
6
PS C:\Users\user1\Desktop\python>
```

Analysis:

Correct selection of a for loop with step size optimization

Logic is clear and efficient

Task Description #2 (AI-Based Code Completion for Loop with Conditionals)

Task: Use an AI code completion tool to combine loops and conditionals.

Prompt:

“Generate Python code to count how many numbers in a list are even and odd.”

Expected Output:

- AI-generated code using loop and if condition.
- Correct count validation.
- Explanation of logic flow.

Prompt:

#Generate Python code to count how many numbers in a list are even and odd using a loop and conditionals.

Sample input list

```
listevenodd.py > ...
1  #Generate Python code to count how many numbers in a list are even and odd using a loop and conditionals.
2  def count_even_odd(numbers):
3      even_count = 0
4      odd_count = 0
5      for num in numbers:
6          if num % 2 == 0:
7              even_count += 1
8          else:
9              odd_count += 1
10     return even_count, odd_count
11
12     # Sample input list
13     numbers_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
14     even_count, odd_count = count_even_odd(numbers_list)
15     print(f"Even numbers: {even_count}")
16     print(f"Odd numbers: {odd_count}"]
```

Output:

```
PS C:\Users\user1\Desktop\python> & C:/Users/user1/AppData/Local/Programs/Python/Python313/python.exe c:/Users/user1/Desktop/python/listevenodd.py
Even numbers: 5
Odd numbers: 5
PS C:\Users\user1\Desktop\python> ]
```

Analysis:

Accurate counting logic

Easy to extend for other classifications

Task Description #3 (AI-Based Code Completion for Class

Attributes Validation)

Task: Use an AI tool to complete a Python class that validates user input.

Prompt:

“Generate a Python class User that validates age and email using conditional statements.”

Expected Output:

- AI-generated class with validation logic.
- Verification of condition handling.
- Test cases for valid and invalid inputs.

Prompt:

#Generate a Python class User that validates age and email using conditional statements.

Test cases

```
ageemail.py > User > validate_age
1 #Generate a Python class User that validates age and email using conditional statements.
2 class User:
3     def __init__(self, age, email):
4         self.age = age
5         self.email = email
6     def validate_age(self):
7         if self.age >= 18:
8             return "Valid age"
9         else:
10            return "Invalid age"
11    def validate_email(self):
12        if "@" in self.email and "." in self.email:
13            return "Valid email"
14        else:
15            return "Invalid email"
16 # Test cases
17 user1 = User(20, "test@example.com")
18 print(user1.validate_age())
19 print(user1.validate_email())
20 user2 = User(15, "testexample.com")
21 print(user2.validate_age())
22 print(user2.validate_email())
```

Output:

```
PS C:\Users\user1\Desktop\python> & C:/Users/user1/AppData/Local/Programs/Python/Python313/python.exe c:/Users/u
ser1/Desktop/python/ageemail.py
Valid age
Valid email
Invalid age
Invalid email
PS C:\Users\user1\Desktop\python> 
```

Analysis:

Correct use of constructor for validation

Clear error handling via exceptions

Task Description #4 (AI-Based Code Completion for Classes)

Task: Use an AI code completion tool to generate a Python class for managing student details.

Prompt:

“Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks.”

Expected Output:

- AI-generated class code.
- Verification of correctness and completeness of class structure.
- Minor manual improvements (if needed) with justification.

Prompt:

#Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks.

Example usage

```

Studentdetails.py > ...
1 #Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks.
2 class Student:
3     def __init__(self, name, roll_number, marks):
4         self.name = name
5         self.roll_number = roll_number
6         self.marks = marks
7
8     def calculate_total(self):
9         return sum(self.marks)
10
11    def calculate_average(self):
12        total = self.calculate_total()
13        return total / len(self.marks) if self.marks else 0
14
15 # Example usage
16 student1 = Student("Alice", 101, [85, 90, 78, 92])
17 print(f"Total marks: {student1.calculate_total()}")
18 print(f"Average marks: {student1.calculate_average()}")

```

Output:

```

PS C:\Users\user1\Desktop\python> & C:/Users/user1/AppData/Local/Programs/Python/Python313/python.exe c:/Users/user1/Desktop/python/Studentdetails.py
Total marks: 345
Average marks: 86.25
PS C:\Users\user1\Desktop\python>

```

Analysis:

Logical separation of data and behavior

Reusability improved by method reuse

Code is readable and maintainable

Task Description 5 (AI-Assisted Code Completion Review)

Task: Use an AI tool to generate a complete Python program using classes, loops, and conditionals together.

Prompt:

“Generate a Python program for a simple bank account system using class, loops, and conditional statements.”

Expected Output:

- Complete AI-generated program.
- Identification of strengths and limitations of AI suggestions.
- Reflection on how AI assisted coding productivity.

Prompt:

#Generate a Python program for a simple bank account system using class, loops, and conditional statements.

Main program

```

bankacc.py > BankAccount > display_balance
1 #Generate a Python program for a simple bank account system using class, loops, and conditional statements.
2 class BankAccount:
3     def __init__(self, name, balance=0):
4         self.name = name
5         self.balance = balance
6     def deposit(self, amount):
7         if amount > 0:
8             self.balance += amount
9             print("Amount deposited successfully")
10        else:
11            print("Invalid deposit amount")
12    def withdraw(self, amount):
13        if amount <= self.balance and amount > 0:
14            self.balance -= amount
15            print("Withdrawal successful")
16        else:
17            print("Invalid or insufficient balance")
18    def display_balance(self):
19        print("Current Balance:", self.balance)
20 # Main program
21 account = BankAccount("Anita", 1000)

```

```

account = BankAccount("Anita", 1000)

while True:
    print("\n1. Deposit\n2. Withdraw\n3. Display Balance\n4. Exit")
    choice = input("Enter your choice: ")

    if choice == "1":
        amount = float(input("Enter amount to deposit: "))
        account.deposit(amount)

    elif choice == "2":
        amount = float(input("Enter amount to withdraw: "))
        account.withdraw(amount)

    elif choice == "3":
        account.display_balance()

    elif choice == "4":
        print("Thank you!")
        break

```

```

bankacc.py > BankAccount > display_balance
29     account.deposit(amount)
30
31     elif choice == "2":
32         amount = float(input("Enter amount to withdraw: "))
33         account.withdraw(amount)
34
35     elif choice == "3":
36         account.display_balance()
37
38     elif choice == "4":
39         print("Thank you!")
40         break
41
42     else:
43         print("Invalid choice")
44

```

Output:

```

PS C:\Users\user1\Desktop\python> & C:/Users/user1/AppData/Local/Programs/Python/Python313/python.exe c:/Users/ser1/Desktop/python/bankacc.py

1. Deposit
2. Withdraw
3. Display Balance
4. Exit
Enter your choice: 1
Enter amount to deposit: 1000
Amount deposited successfully

1. Deposit
2. Withdraw
3. Display Balance
4. Exit
Enter your choice: 2
Enter amount to withdraw: 500
Withdrawal successful

1. Deposit
2. Withdraw
3. Display Balance
4. Exit
Enter your choice: 3
Current Balance: 1500.0

1. Deposit
2. Withdraw
3. Display Balance
4. Exit
Enter your choice: 4
Thank you!
PS C:\Users\user1\Desktop\python>

```

Analysis:

Real-world inspired example

Demonstrates program flow clearly

